mlp_week10

March 26, 2026

1 Week 10 - Introduction to Neural Networks: Part II

Yuting Xie (preferred: Ella Xie)

1.0.1 Aims

The main concepts covered in this notebook are:

- getting familiar with stochastic gradient descent and the role of the learning rate and batch size
- ways to cope with overfitting, including early stopping, regularization, and dropout
- incorporating the keras model into an sklearn pipeline

It may be helpful to have last week's notebook open to recall how to build, compile, and fit neural networks in keras.

When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

First, we need to install the versions of tensorflow and keras needed for this workshop. The versions in Notable's Machine Learning and Language notebook are older, so anyone using Noteable, should start a Python 3 notebook and then uncomment the lines below to install the correct versions.

[1]:
```
!pip install tensorflow==2.20.0
!pip install tf-keras==2.20.1
```

```
Collecting tensorflow==2.20.0
  Using cached tensorflow-2.20.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow==2.20.0)
  Using cached absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
```

```
Collecting astunparse>=1.6.0 (from tensorflow==2.20.0)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow==2.20.0)
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow==2.20.0)
  Using cached gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow==2.20.0)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow==2.20.0)
  Using cached libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl.metadata
(5.2 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow==2.20.0)
  Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in /opt/conda/lib/python3.12/site-
packages (from tensorflow==2.20.0) (25.0)
Requirement already satisfied: protobuf>=5.28.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow==2.20.0) (5.29.3)
Requirement already satisfied: requests<3,>=2.21.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow==2.20.0) (2.32.5)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-
packages (from tensorflow==2.20.0) (80.9.0)
Requirement already satisfied: six>=1.12.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow==2.20.0) (1.17.0)
Collecting termcolor>=1.1.0 (from tensorflow==2.20.0)
  Using cached termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in
/opt/conda/lib/python3.12/site-packages (from tensorflow==2.20.0) (4.14.0)
Collecting wrapt>=1.11.0 (from tensorflow==2.20.0)
  Using cached wrapt-2.1.2-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (7.4
kB)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/opt/conda/lib/python3.12/site-packages (from tensorflow==2.20.0) (1.71.0)
Collecting tensorboard~=2.20.0 (from tensorflow==2.20.0)
  Using cached tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
Collecting keras>=3.10.0 (from tensorflow==2.20.0)
  Using cached keras-3.13.2-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow==2.20.0) (1.26.4)
Requirement already satisfied: h5py>=3.11.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow==2.20.0) (3.15.1)
Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow==2.20.0)
  Using cached ml_dtypes-0.5.4-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.9 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow==2.20.0) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-
```

```
packages (from requests<3,>=2.21.0->tensorflow==2.20.0) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow==2.20.0) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow==2.20.0) (2025.6.15)
Requirement already satisfied: markdown>=2.6.8 in
/opt/conda/lib/python3.12/site-packages (from
tensorboard~=2.20.0->tensorflow==2.20.0) (3.10.1)
Requirement already satisfied: pillow in /opt/conda/lib/python3.12/site-packages
(from tensorboard~=2.20.0->tensorflow==2.20.0) (10.4.0)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from
tensorboard~=2.20.0->tensorflow==2.20.0)
  Using cached tensorboard_data_server-0.7.2-py3-none-
manylinux_2_31_x86_64.whl.metadata (1.1 kB)
Collecting werkzeug>=1.0.1 (from tensorboard~=2.20.0->tensorflow==2.20.0)
  Using cached werkzeug-3.1.7-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/opt/conda/lib/python3.12/site-packages (from
astunparse>=1.6.0->tensorflow==2.20.0) (0.45.1)
Requirement already satisfied: rich in /opt/conda/lib/python3.12/site-packages
(from keras>=3.10.0->tensorflow==2.20.0) (14.3.2)
Collecting namex (from keras>=3.10.0->tensorflow==2.20.0)
  Using cached namex-0.1.0-py3-none-any.whl.metadata (322 bytes)
Requirement already satisfied: optree in /opt/conda/lib/python3.12/site-packages
(from keras>=3.10.0->tensorflow==2.20.0) (0.18.0)
Requirement already satisfied: markupsafe>=2.1.1 in
/opt/conda/lib/python3.12/site-packages (from
werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow==2.20.0) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/opt/conda/lib/python3.12/site-packages (from
rich->keras>=3.10.0->tensorflow==2.20.0) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/conda/lib/python3.12/site-packages (from
rich->keras>=3.10.0->tensorflow==2.20.0) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.12/site-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow==2.20.0)
(0.1.2)
Using cached
tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(620.7 MB)
Using cached
ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (5.0
MB)
Using cached tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
Using cached tensorboard_data_server-0.7.2-py3-none-manylinux_2_31_x86_64.whl
(6.6 MB)
```

```
Using cached absl_py-2.4.0-py3-none-any.whl (135 kB)
Using cached astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Using cached flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Using cached gast-0.7.0-py3-none-any.whl (22 kB)
Using cached google_pasta-0.2.0-py3-none-any.whl (57 kB)
Using cached keras-3.13.2-py3-none-any.whl (1.5 MB)
Using cached libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl (24.5 MB)
Using cached opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Using cached termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Using cached werkzeug-3.1.7-py3-none-any.whl (226 kB)
Using cached wrapt-2.1.2-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (121 kB)
Using cached namex-0.1.0-py3-none-any.whl (5.9 kB)
Installing collected packages: namex, libclang, flatbuffers, wrapt, werkzeug,
termcolor, tensorboard-data-server, opt_einsum, ml_dtypes, google_pasta, gast,
astunparse, absl-py, tensorboard, keras, tensorflow
                          16/16
[tensorflow]6 [tensorflow]]]ata-server]
Successfully installed absl-py-2.4.0 astunparse-1.6.3
flatbuffers-25.12.19 gast-0.7.0 google_pasta-0.2.0 keras-3.13.2 libclang-18.1.1
ml_dtypes-0.5.4 namex-0.1.0 opt_einsum-3.4.0 tensorboard-2.20.0 tensorboard-
data-server-0.7.2 tensorflow-2.20.0 termcolor-3.3.0 werkzeug-3.1.7 wrapt-2.1.2
Collecting tf-keras==2.20.1
  Using cached tf_keras-2.20.1-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: tensorflow<2.21,>=2.20 in
/opt/conda/lib/python3.12/site-packages (from tf-keras==2.20.1) (2.20.0)
Requirement already satisfied: absl-py>=1.0.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (2.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (3.4.0)
Requirement already satisfied: packaging in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (25.0)
```

```
Requirement already satisfied: protobuf>=5.28.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (5.29.3)
Requirement already satisfied: requests<3,>=2.21.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (2.32.5)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (80.9.0)
Requirement already satisfied: six>=1.12.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (4.14.0)
Requirement already satisfied: wrapt>=1.11.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (1.71.0)
Requirement already satisfied: tensorboard~=2.20.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (1.26.4)
Requirement already satisfied: h5py>=3.11.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow<2.21,>=2.20->tf-keras==2.20.1) (3.15.1)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in
/opt/conda/lib/python3.12/site-packages (from tensorflow<2.21,>=2.20->tf-
keras==2.20.1) (0.5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-
packages (from requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1)
(3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
/opt/conda/lib/python3.12/site-packages (from
requests<3,>=2.21.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (2025.6.15)
Requirement already satisfied: markdown>=2.6.8 in
/opt/conda/lib/python3.12/site-packages (from
tensorboard~=2.20.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (3.10.1)
```

```
Requirement already satisfied: pillow in /opt/conda/lib/python3.12/site-packages
(from tensorboard~=2.20.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (10.4.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/opt/conda/lib/python3.12/site-packages (from
tensorboard~=2.20.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/opt/conda/lib/python3.12/site-packages (from
tensorboard~=2.20.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (3.1.7)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/opt/conda/lib/python3.12/site-packages (from
astunparse>=1.6.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (0.45.1)
Requirement already satisfied: rich in /opt/conda/lib/python3.12/site-packages
(from keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (14.3.2)
Requirement already satisfied: namex in /opt/conda/lib/python3.12/site-packages
(from keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (0.1.0)
Requirement already satisfied: optree in /opt/conda/lib/python3.12/site-packages
(from keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (0.18.0)
Requirement already satisfied: markupsafe>=2.1.1 in
/opt/conda/lib/python3.12/site-packages (from
werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1)
(3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/opt/conda/lib/python3.12/site-packages (from
rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/conda/lib/python3.12/site-packages (from
rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.12/site-
packages (from markdown-it-
py>=2.2.0->rich->keras>=3.10.0->tensorflow<2.21,>=2.20->tf-keras==2.20.1)
(0.1.2)
Using cached tf_keras-2.20.1-py3-none-any.whl (1.7 MB)
Installing collected packages: tf-keras
Successfully installed tf-keras-2.20.1
```

2.1 Packages

Let's load the some of the packages needed for this workshop.

[2]:
```
# Import necessary libraries
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
2026-03-26 16:14:39.557780: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
2026-03-26 16:14:39.558123: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-03-26 16:14:39.735324: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other
operations, rebuild TensorFlow with the appropriate compiler flags.
2026-03-26 16:14:49.586841: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-03-26 16:14:49.587462: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
```

3 Part 1: Motorcycle Data

In the first part of this workshop, we will work with the motorcyle data, which giving a series of measurements of head acceleration in a simulated motorcycle accident, used to test crash helmets.

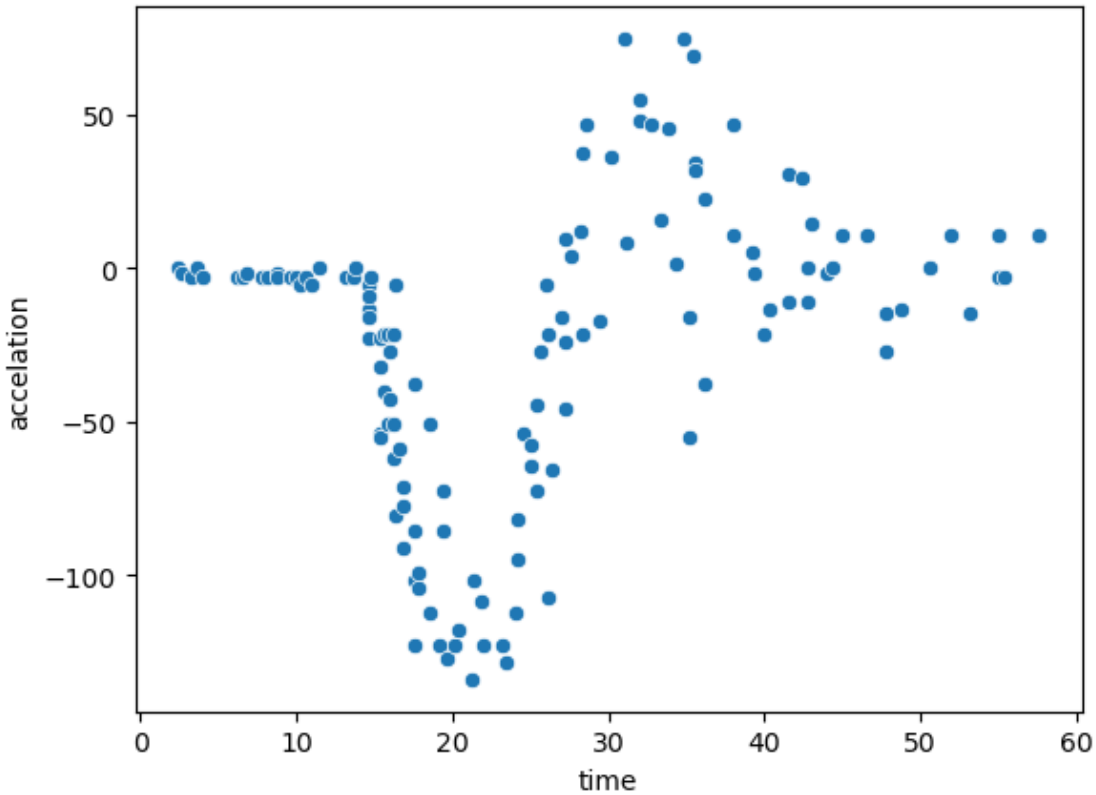
This is a small data set with only 1 feature (time) and 1 numerical outcome (head acceleration) recorded at $N = 133$ time points, but we will use it illustrate the role of some of parameters in neural networks and gradient descent.

[3]:
```
# Load motorcycle data
mcycle = pd.read_csv('mcycle.csv', index_col=0)
N = mcycle.shape[0]
D = mcycle.shape[1]-1
y = mcycle['accel']
X = mcycle.drop('accel', axis=1)
print('Number of observations: ', N)

# Scale the inputs
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
X = ss.fit_transform(X)

# Set seed for reproducibility
keras.utils.set_random_seed(11205)
```

```
Number of observations:  133
```

[4]:
```
# Plot the data
sns.scatterplot(mcycle, x = 'times', y = 'accel')
plt.xlabel('time')
```

```
plt.ylabel('accelation')
plt.show()
```



3.0.1 Exercise 1 (CORE)

Define a neural network in keras with two hidden layers, the first with 20 hidden units and the second with 10 hiddent units, and ReLU activations. You should call your model `model_reg`.

How many parameters does the model have?

Note: since this is a regression task, the final activation should be the identity (`linear` function in keras).

[5]:
```
# Code for your answer here!
# Name your model: model_reg
model_reg = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(D,)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model_reg.summary()
```

```
/opt/conda/lib/python3.12/site-packages/keras/src/layers/core/dense.py:106:
```

```
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-03-26 16:14:51.535080: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 20)	40
dense_1 (Dense)	(None, 10)	210
dense_2 (Dense)	(None, 1)	11

Total params: 261 (1.02 KB)

Trainable params: 261 (1.02 KB)

Non-trainable params: 0 (0.00 B)

we have 261 parameters.

3.0.2 Compile the model

Now, let's compile the model. Here, we are using - the mean squared error loss - the SGD optimizer - computing additional metrics, namely R2 and MAE.

[6]:
```
model_reg.compile(
    loss=keras.losses.MeanSquaredError(),
    optimizer= keras.optimizers.SGD(learning_rate=1e-3),
    metrics=[
       keras.metrics.R2Score(name='R2'),
       keras.metrics.MeanAbsoluteError(name='MAE')
    ],
)
```

Before, we fit the data, we will first shuffle it. Note the data here is stored in order of time. If we did not first shuffle, the validation would look very different (and consist of the final time points).

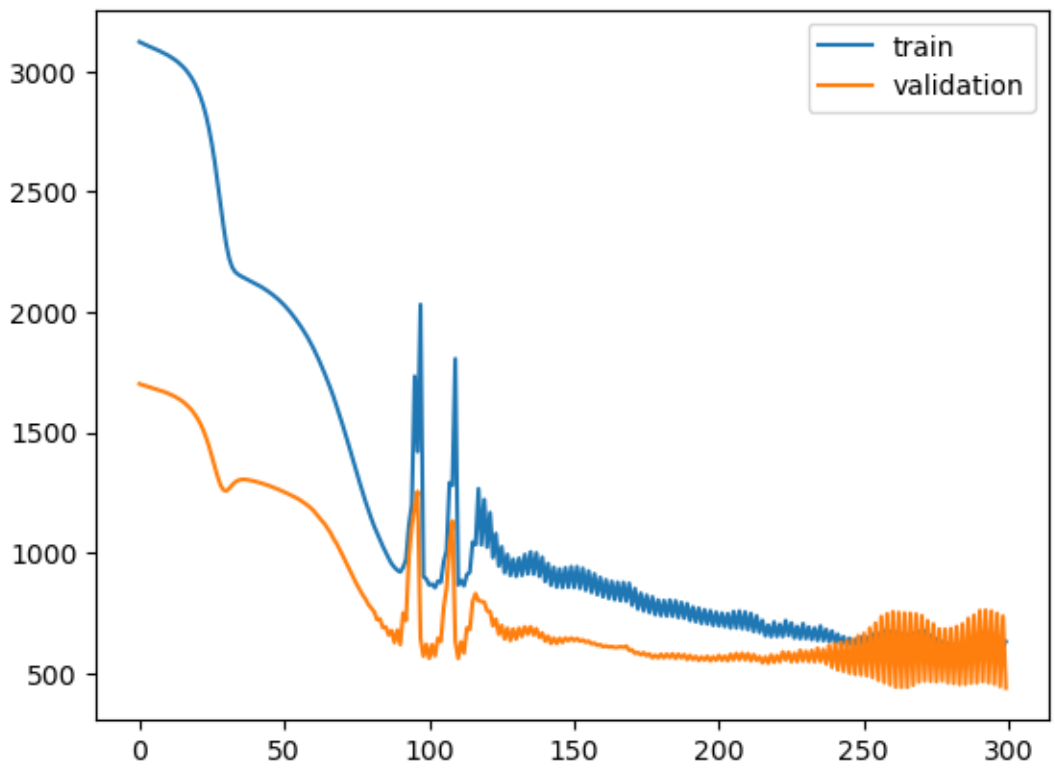
[7]:
```
from sklearn.utils import shuffle
X, y = shuffle(X, y, random_state=11205)
```

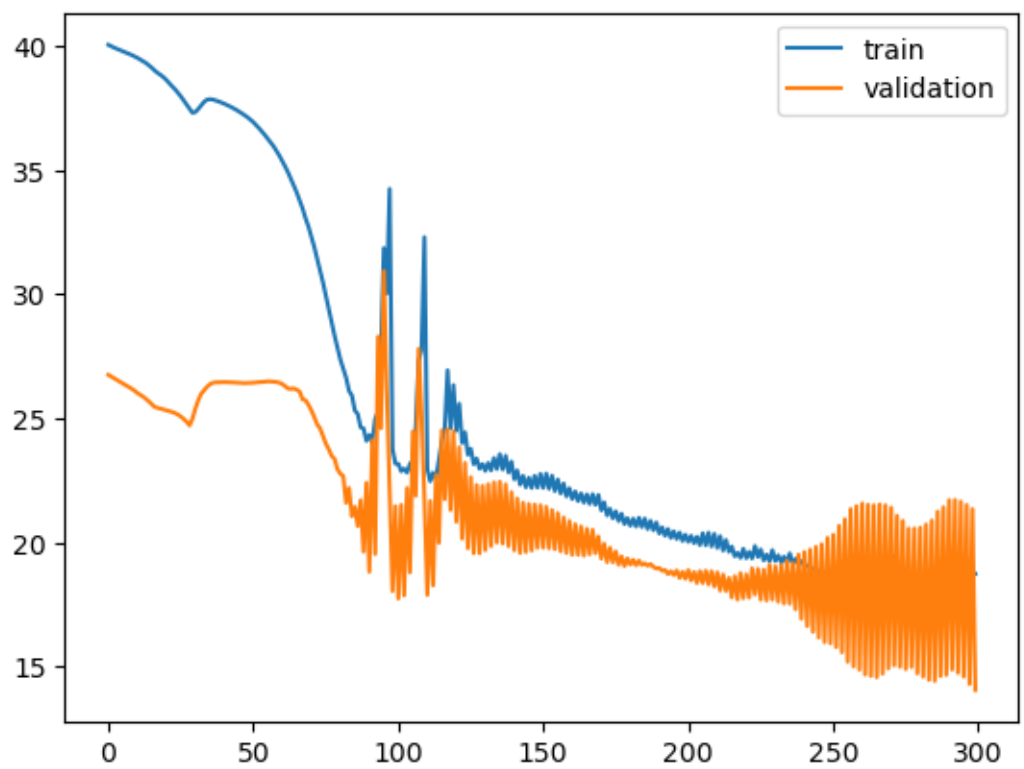
3.0.3 Exercise 2 (CORE)

Fit the model with the following options: - 300 epochs - batch size equal to N (hint: set the optional parameter `batch_size` in `.fit()`) - validation set to 10% of the data - you may also want to set `verbose=0` to avoid printing the output

Plot the metrics as functions of the number of epochs. Comment on their behavior and if you think the model should be trained for more or less epochs.

[8]:
```
# Code for your answer here!
history=model_reg.fit(
    X,y,
    epochs=300,
    batch_size=N,
    validation_split=0.1,
    verbose=0
)
plt.figure()
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='validation')
plt.legend()
plt.show()
plt.figure()
plt.plot(history.history['MAE'], label='train')
plt.plot(history.history['val_MAE'], label='validation')
plt.legend()
plt.show()
```


train
validation
3000
2500
2000
1500
1000
500
0
50
100
150
200
250
300

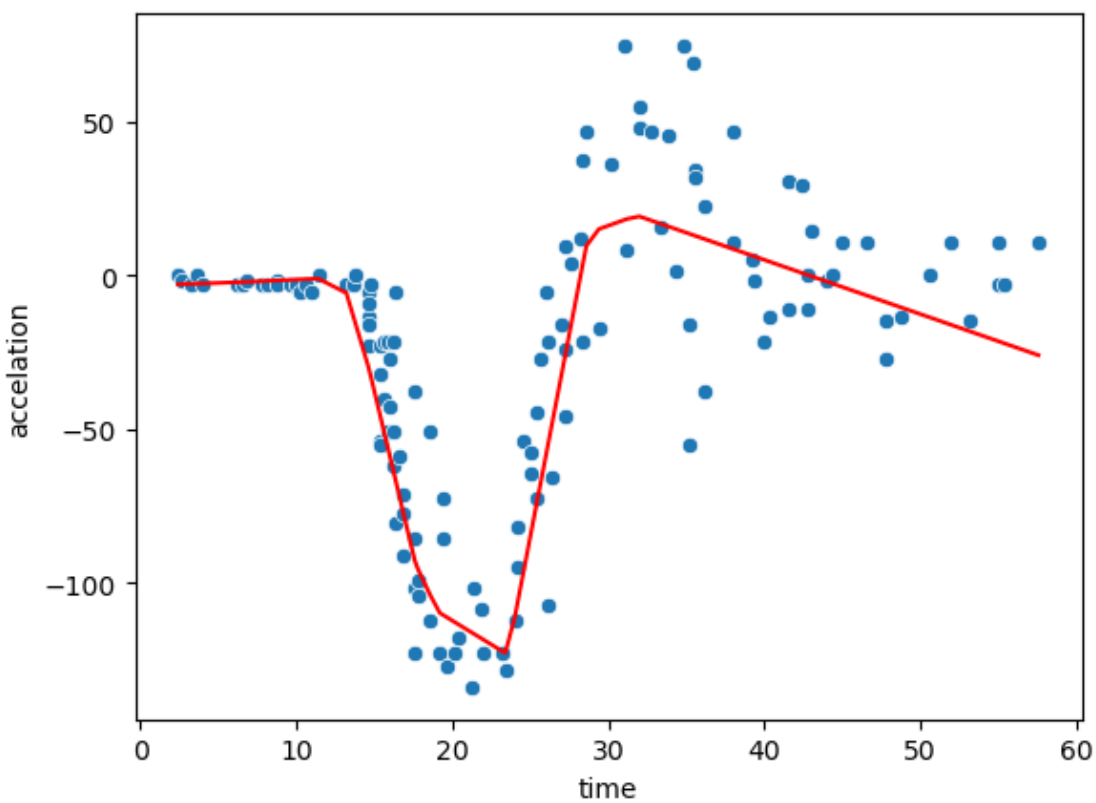

train
validation
40
35
30
25
20
15
0
50
100
150
200
250
300

the validation loss is very unstable towards later epoch. thus we need less epochs.

Let's plot the predictions of the fitted model.

[9]:
```
# Compute predictions
yhat = model_reg.predict(X)

# Add them to the plot
sns.scatterplot(mcycle, x = 'times', y = 'accel')
sns.lineplot(x = ss.inverse_transform(X).reshape(-1), y = yhat.reshape(-1), c =␣
 ↪'red')
plt.xlabel('time')
plt.ylabel('accelation')
plt.show()
```

```
5/5              0s 14ms/step
```



3.0.4 Exercise 3 (CORE): Exploring the role of the learning rate

Repeat the above but change the learning rate to

a) `learning_rate=1e-2` and call this `model_reg2`

b) `learning_rate=1e-4` and call this `model_reg3`

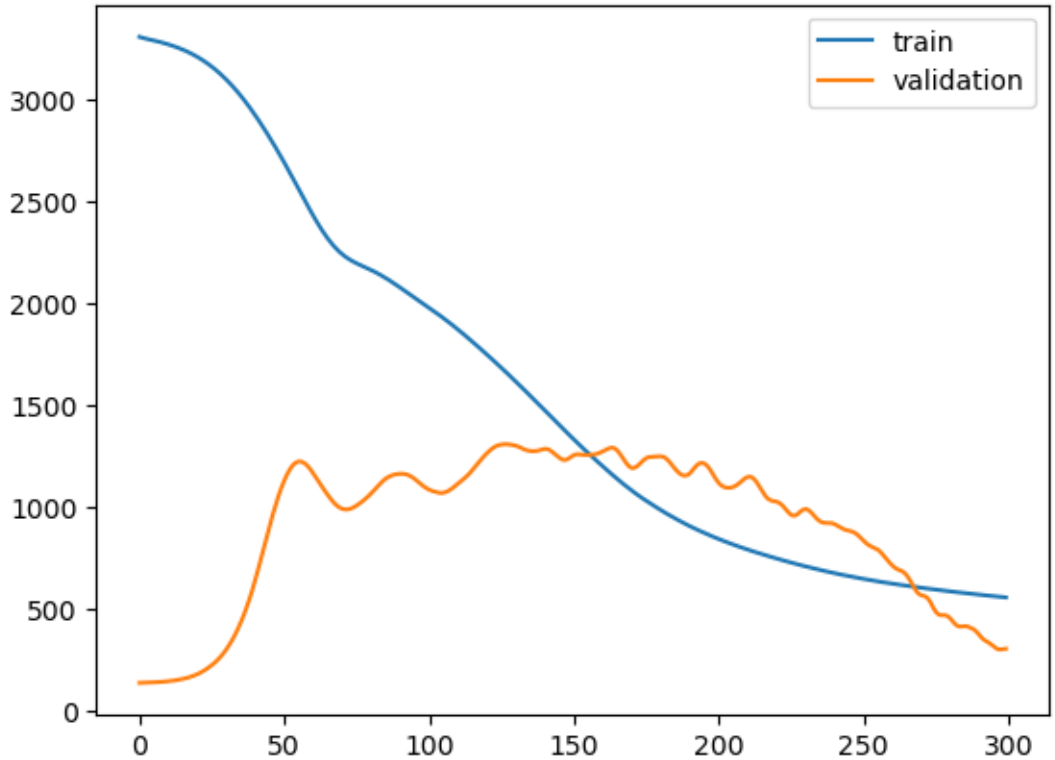
What is the effect of the learning rate? - Comment on the behavior of the metrics as a function epochs for these choices compared with the previous. - Plot also the predictions and comment on they compare with the previous model fit.

Note: redefine the model before compiling so that the biases and weights are re-initialized randomly.

[10]:
```
# Option a: larger learning rateprint(X.dtype)
# Code for your answer here!
mcycle = pd.read_csv('mcycle.csv', index_col=0)
y = mcycle['accel']
X = mcycle.drop('accel', axis=1)
N = X.shape[0]
D = X.shape[1]
X = pd.get_dummies(X, drop_first=True)
ss = StandardScaler()
X = ss.fit_transform(X)
X = np.array(X, dtype=float)
y = np.array(y, dtype=float)

model_reg2 = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model_reg2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-2),
    loss='mse',
    metrics=['mae']
)
history2 = model_reg2.fit(
    X, y,
    epochs=300,
    batch_size=N,
    validation_split=0.1,
    verbose=0
)

plt.figure()
plt.plot(history2.history['loss'], label='train')
plt.plot(history2.history['val_loss'], label='validation')
plt.legend()
plt.show()
```

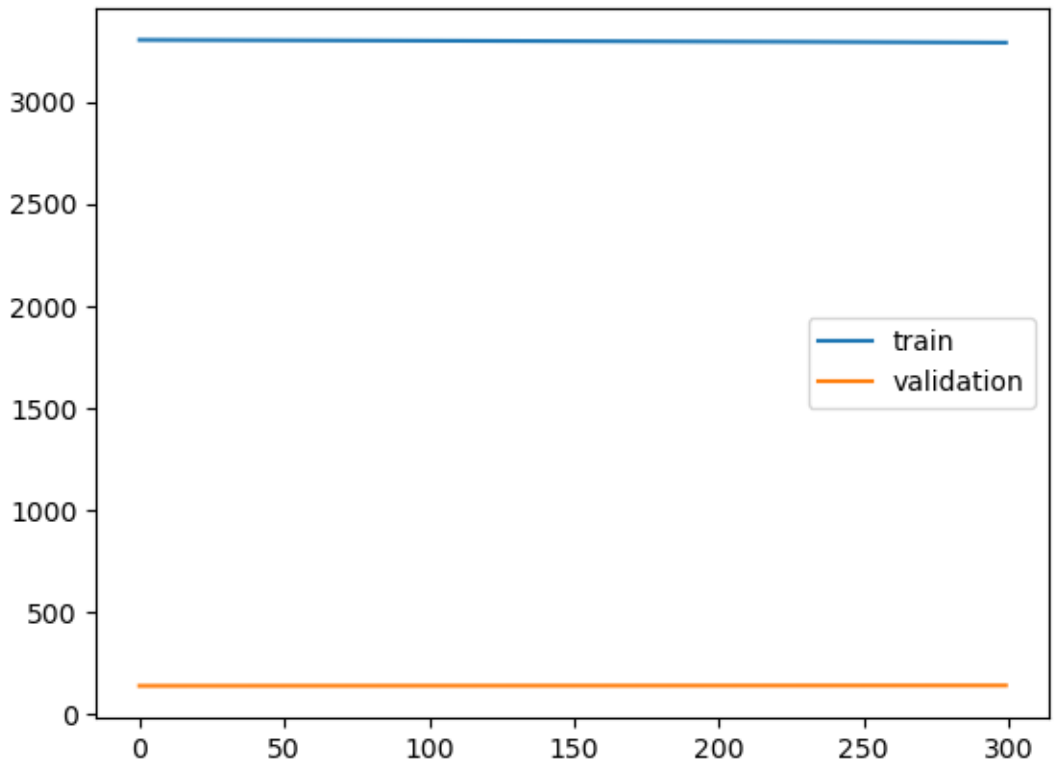
```
/opt/conda/lib/python3.12/site-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
```

```
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```



[11]:
```
# Option b: smaller learning rate

# Code for your answer here!
model_reg3 = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model_reg3.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='mse',
    metrics=['mae']
)
history3 = model_reg3.fit(
    X, y,
    epochs=300,
    batch_size=N,
    validation_split=0.1,
    verbose=0
```

```
)

plt.figure()
plt.plot(history3.history['loss'], label='train')
plt.plot(history3.history['val_loss'], label='validation')
plt.legend()
plt.show()
```



if the learning is very small, the model almost learn nothing.

3.0.5 Exercise 4 (CORE): Exploring the effect of the batch size

Previously we used gradient descent (`batch_size=N`). Consider changing the batch size to:

a) `batch_size=20` and call this `model_reg4`

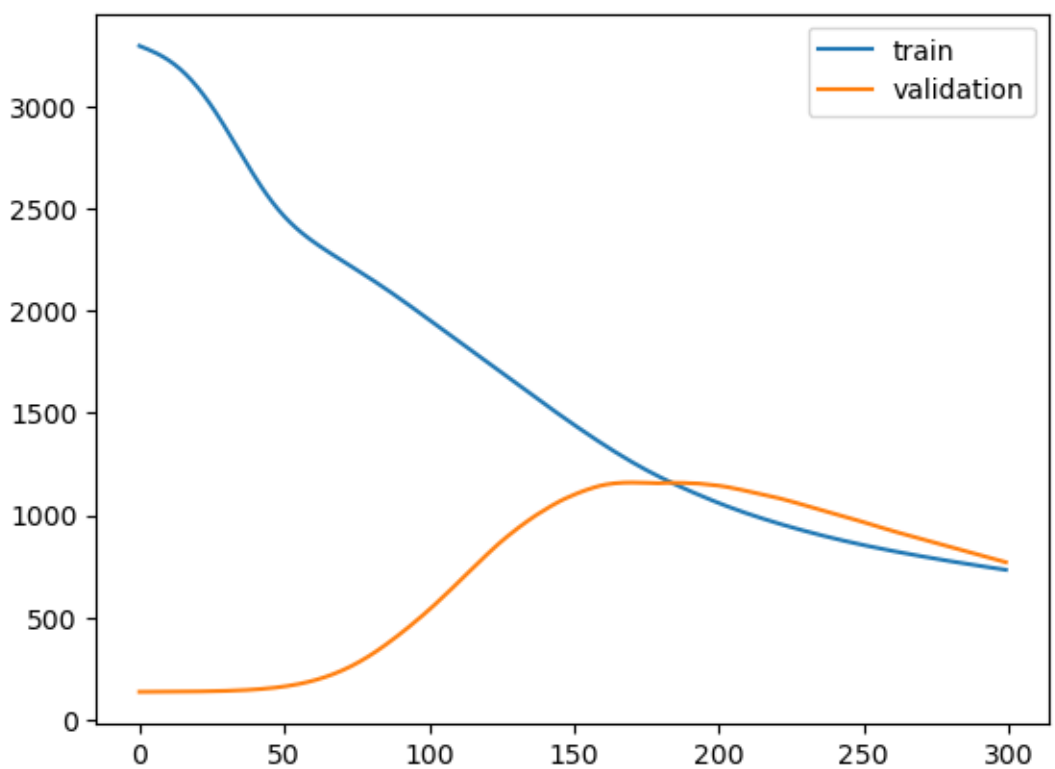
b) `batch_size=1` and call this `model_reg5`

Comment on the effect of the batch size, and your preferred choice.

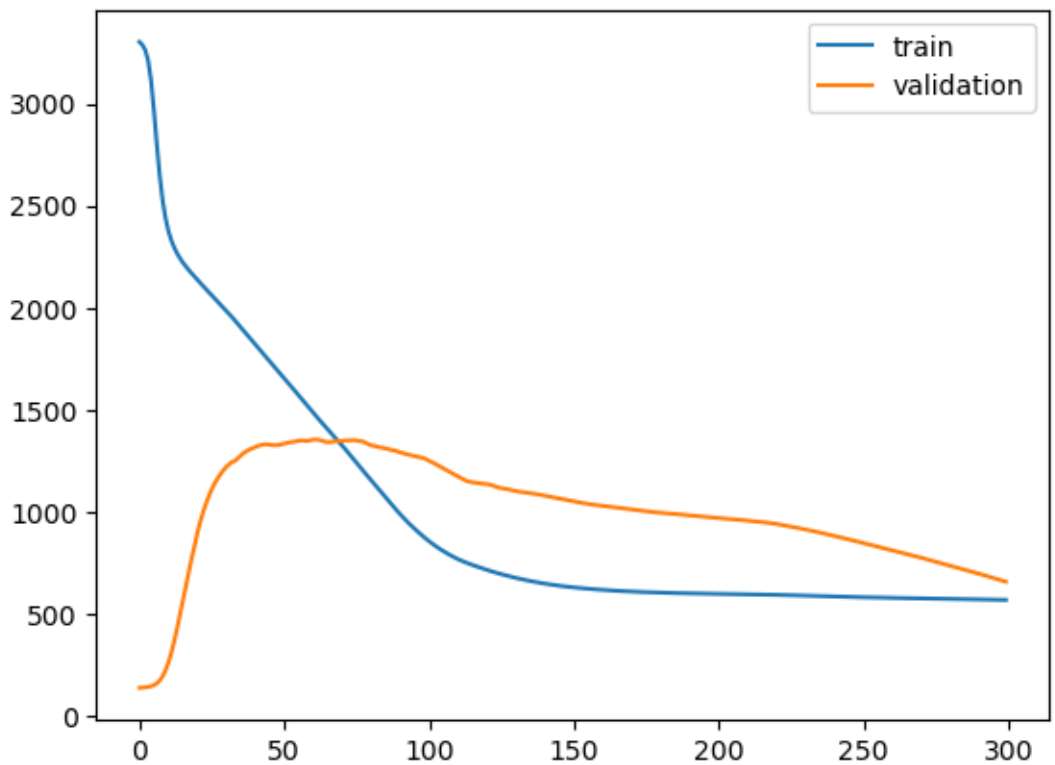
Note: redefine the model before compiling so that the biases and weights are re-initialized randomly.

[12]:
```
# Option a: batch size of 20

# Code for your answer here!
```

```
model_reg4 = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model_reg4.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',
    metrics=['mae']
)
history4 = model_reg4.fit(
    X, y,
    epochs=300,
    batch_size=20,
    validation_split=0.1,
    verbose=0
)
plt.figure()
plt.plot(history4.history['loss'], label='train')
plt.plot(history4.history['val_loss'], label='validation')
plt.legend()
plt.show()
```



[13]:
```
# Option b: batch size of 1

# Code for your answer here!
model_reg5 = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model_reg5.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',
    metrics=['mae']
)
history5 = model_reg5.fit(
    X, y,
    epochs=300,
    batch_size=1,
    validation_split=0.1,
    verbose=0
)
plt.figure()
plt.plot(history5.history['loss'], label='train')
plt.plot(history5.history['val_loss'], label='validation')
plt.legend()
plt.show()
```



small batch size means catch more noise give more unstable output. large batch size perform well when we have more epoch.

3.0.6 Early stopping

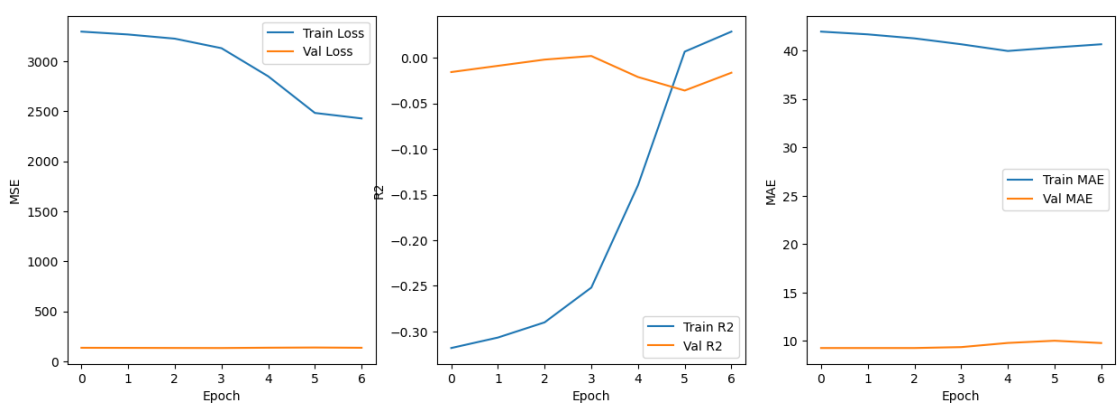
When using minibatches, SGD converges much faster and 300 epochs seems unnecessary. We can use keras's EarlyStopping to exit early. In this case, we need to decide: - the metric to monitor (`monitor`) - the minimum change in the monitored quantity to qualify as an improvement (`min_delta`) - the number of epochs with no improvement after which training will be stopped (`patience`)

In the following, we use early stopping to avoid training for the full 300 epochs.

[14]:
```
# Defining the layers of the model
input_layer = keras.layers.Input(shape=(D,))
hidden_layer1 = keras.layers.Dense(20, activation='relu') (input_layer)
hidden_layer2 = keras.layers.Dense(10, activation='relu') (hidden_layer1)
output_layer = keras.layers.Dense(1, activation='linear') (hidden_layer2)
model_reg4 = keras.models.Model(input_layer, output_layer)

model_reg4.compile(
    loss=keras.losses.MeanSquaredError(),
    optimizer= keras.optimizers.SGD(learning_rate=1e-3),
```

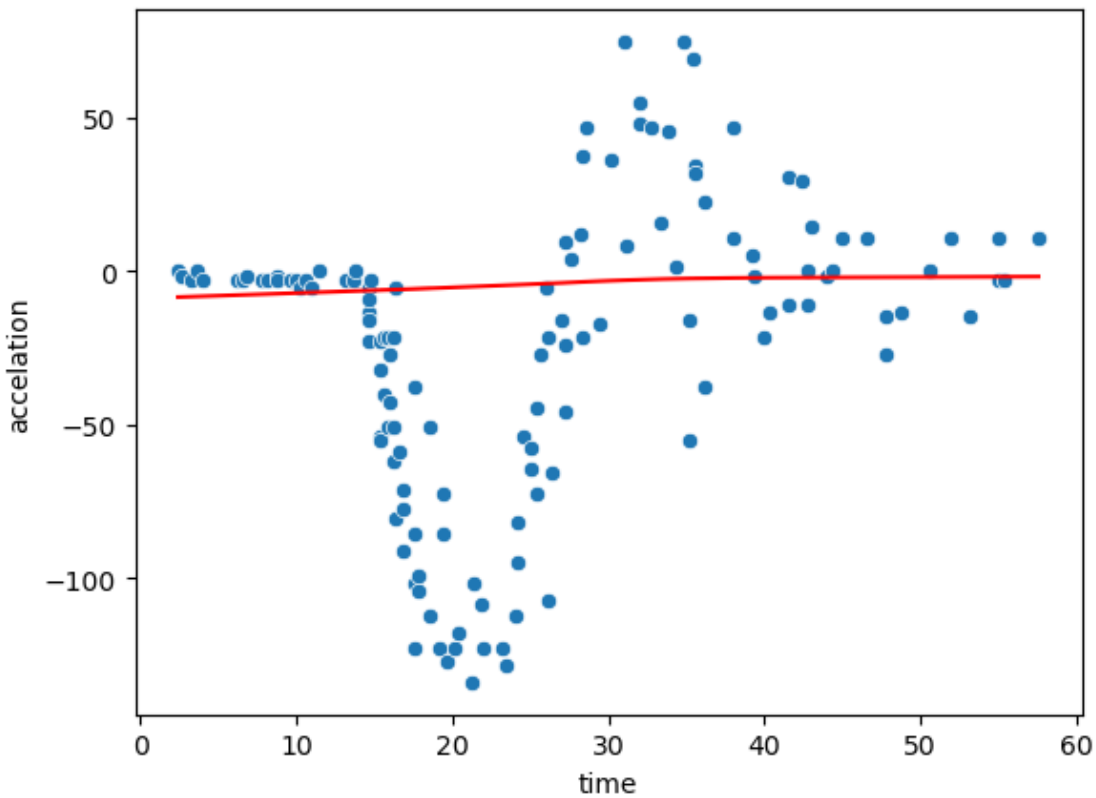
```
    metrics=[
        keras.metrics.R2Score(name='R2'),
        keras.metrics.MeanAbsoluteError(name='MAE')
    ],
)
```

[15]:
```
# Fit the model
callback = keras.callbacks.EarlyStopping(monitor='val_loss', min_delta = 0,␣
 ↪mode = 'min',
                                         patience=3, restore_best_weights=True)

model_reg4.fit(x=X, y=y, epochs=300, shuffle=True, validation_split=0.1,␣
 ↪verbose=0, batch_size=20, callbacks=[callback])

# Plot the training history
history = model_reg4.history.history
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].plot(history['loss'], label='Train Loss')
ax[0].plot(history['val_loss'], label='Val Loss')
ax[0].set_xlabel('Epoch')
ax[0].set_ylabel('MSE')
ax[0].legend()
ax[1].plot(history['R2'], label='Train R2')
ax[1].plot(history['val_R2'], label='Val R2')
ax[1].set_xlabel('Epoch')
ax[1].set_ylabel('R2')
ax[1].legend()
ax[2].plot(history['MAE'], label='Train MAE')
ax[2].plot(history['val_MAE'], label='Val MAE')
ax[2].set_xlabel('Epoch')
ax[2].set_ylabel('MAE')
ax[2].legend()
plt.show()
```



[16]:
```
# Compute predictions
yhat = model_reg4.predict(X)

# Add them to the plot
sns.scatterplot(mcycle, x = 'times', y = 'accel')
sns.lineplot(x = ss.inverse_transform(X).reshape(-1), y = yhat.reshape(-1), c =␣
 ↪'red')
plt.xlabel('time')
plt.ylabel('accelation')
plt.show()
```

```
5/5              0s 11ms/step
```



But, it seems that it has stopped too early!

3.0.7 Exercise 5 (CORE)

Try changing any or all of the three parameters determining the early stopping criteria. What have you decided and why? How has it changed the number of epochs before exiting?

[47]:
```
# Code for your answer here!
from tensorflow.keras.callbacks import EarlyStopping
X = pd.get_dummies(X, drop_first=True)
ss = StandardScaler()
X = ss.fit_transform(X)
X = np.array(X, dtype=float)
y = np.array(y, dtype=float)
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    min_delta=1e-3,
    restore_best_weights=True
)
model = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',
    metrics=['mae']
)
history = model.fit(
    X, y,
    epochs=300,
    batch_size=20,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=0
)
len(history.history['loss'])
```

```
/opt/conda/lib/python3.12/site-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[47]: 207

207 epochs. i set a small patience to get the proper number of epoch.

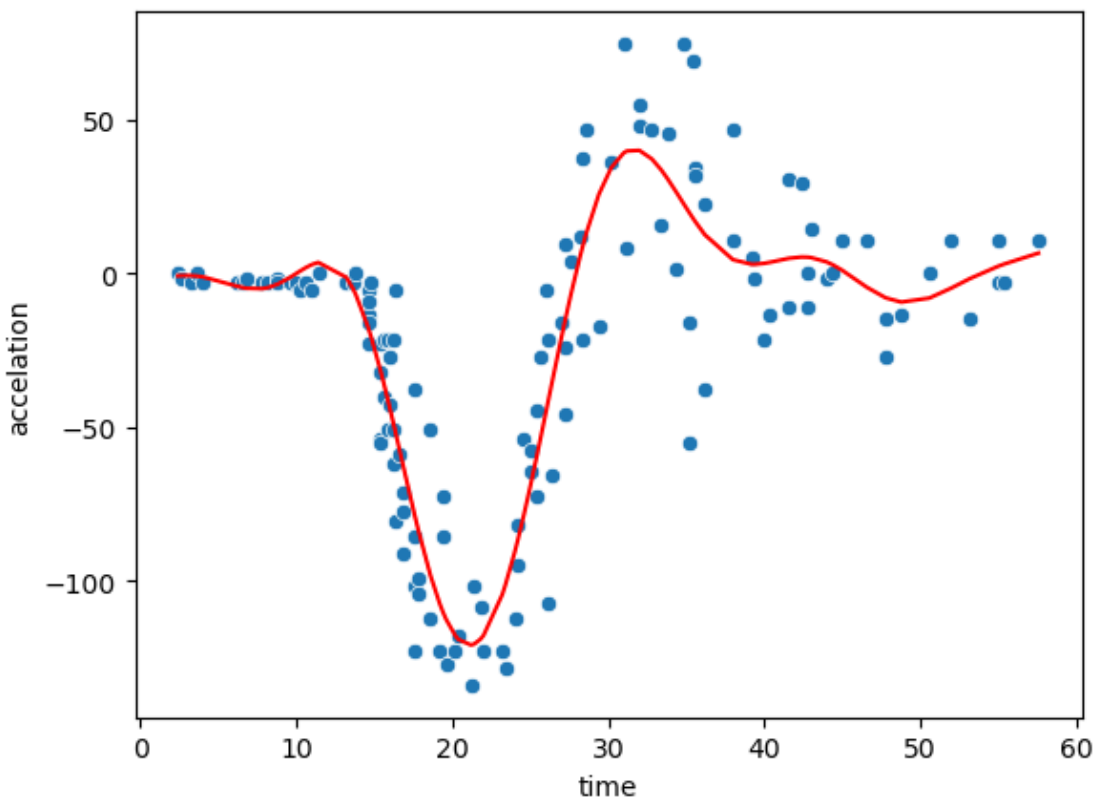
3.0.8 Exercise 6 (EXTRA)

For comparison, let's fit kernel ridge regression model with radial basis function.

- Try changing the value of gamma in the code below. How does it effect the regression function?
- Try changing the value of alpha in the code below. How does it effect the regression function?

- Comment on which model you prefer (neural network or kernel ridge) both in terms of predictive performance and ease of implementation.

[18]:
```
from sklearn.kernel_ridge import KernelRidge

# Define and fit the model
kr = KernelRidge(alpha = 0.1, kernel='rbf', gamma=4).fit(X,y)

# Compute predictions
yhat = kr.predict(X)

# Add them to the plot
sns.scatterplot(mcycle, x = 'times', y = 'accel')
sns.lineplot(x = ss.inverse_transform(X).reshape(-1), y = yhat, c = 'red')
plt.xlabel('time')
plt.ylabel('accelation')
plt.show()
```



Type your answer here

3.0.9 Heteroskedastic regression with a custom loss

Our previous model only provided an estimate of the mean function for acceleration as a function of time. However, visually, it is evident that the variance of acceleration also varies with time.

Let's define a custom loss function for the case of heteroskedastic regression. In this case, the loss corresponds to the negative log likelihood of the Gaussian with unknown mean and standard deviation (see Lecture 9 Notes). Here, our neural network model will output both the mean and standard deviation, we have concantenated them together to output an matrix with two columns (the first for the mean, and the second for the standard deviation).

[19]:
```
# Gaussian NLL as a custom loss
def my_loss(y_true, y_pred):
    mu_pred = y_pred[:, 0]
    sigma_pred = y_pred[:, 1]
    y_true = tf.cast(y_true, mu_pred.dtype)
    y_true = tf.reshape(y_true, tf.shape(mu_pred))

    # NLL (up to a constant): 0.5 * ((y - mu)/sigma)^2 + log(sigma)
    sq_term = 0.5 * tf.square((y_true - mu_pred) / sigma_pred)
    log_term = tf.math.log(sigma_pred)
    nll = sq_term + log_term

    # Reduce mean over batch
    return tf.reduce_mean(nll)
```

Now, let's start with a simple model for illustration. We will keep the same architecture for the mean, but for the standard deviation, for simplicity, we assume no hidden layer (it depends only on the inputs directly, i.e. subject to the nonlinear activation, there is a linear relation between time and the standard deviation of acceleration).

For the ouput of the standard deviaton, we use the `softplus` activation to ensure that it is positive.

[20]:
```
# Defining the layers of the model
input_layer = keras.layers.Input(shape=(D,))
hidden_layer1 = keras.layers.Dense(20, activation='relu') (input_layer)
hidden_layer2 = keras.layers.Dense(10, activation='relu') (hidden_layer1)
output_mean = keras.layers.Dense(1, activation='linear') (hidden_layer2)
output_sigma = keras.layers.Dense(1, activation='softplus') (input_layer)
stack = keras.layers.Concatenate()([output_mean, output_sigma])
model_reg_het = keras.Model(input_layer, stack)

model_reg_het.compile(
    loss=my_loss,
    optimizer= keras.optimizers.SGD(learning_rate=0.001),
    metrics=[
       keras.metrics.R2Score(name='R2'),
       keras.metrics.MeanAbsoluteError(name='MAE')
    ],
```

```
)
```

3.0.10 Exercise 7 (CORE)

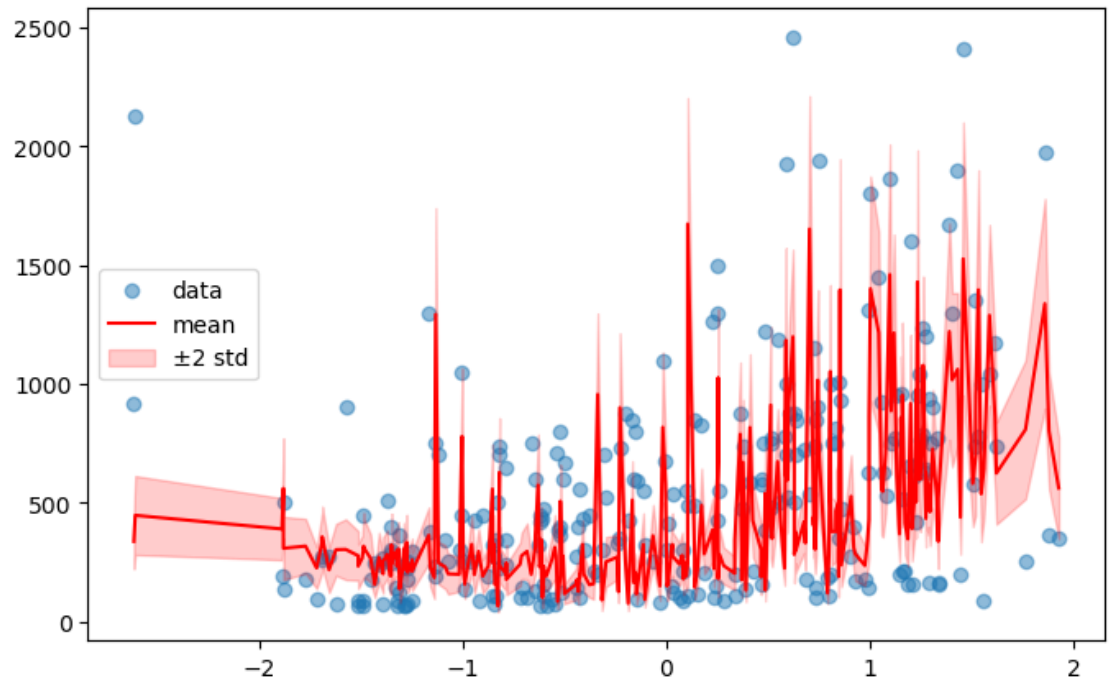
Train the model for as many epochs as needed (when early stopping exits), and plot the mean function, also with lines for +/- 2*standard deviation, overlaid on top of a scatterplot of the data.

How does the model perform?

Note: if it exits without training, re-initialize by running the code cell above again.

[50]:
```
# Fit the model
model = keras.Sequential([
    layers.Dense(20, activation='relu', input_shape=(X.shape[1],)),
    layers.Dropout(0.1),
    layers.Dense(10, activation='relu'),
    layers.Dropout(0.1),
    layers.Dense(1, activation='linear')
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',
    metrics=['mae']
)
callback = keras.callbacks.EarlyStopping(monitor='loss', min_delta = 0.05, mode␣
 ↪='min',
                                         patience=10, restore_best_weights=True)

# Code for your answer here!
history = model.fit(
    X, y,
    epochs=300,
    batch_size=20,
    validation_split=0.1,
    callbacks=[callback],
    verbose=0
)

T = 100
preds = np.array([model(X, training=True).numpy().flatten() for _ in range(T)])
mean_pred = preds.mean(axis=0)
std_pred = preds.std(axis=0)
idx = np.argsort(X[:, 0])
X_sorted = X[idx]
y_sorted = y[idx]
mean_sorted = mean_pred[idx]
std_sorted = std_pred[idx]
plt.figure(figsize=(8,5))
```

```
plt.scatter(X[:,0], y, alpha=0.5, label='data')
plt.plot(X_sorted[:,0], mean_sorted, color='red', label='mean')
plt.fill_between(
    X_sorted[:,0],
    mean_sorted - 2*std_sorted,
    mean_sorted + 2*std_sorted,
    color='red',
    alpha=0.2,
    label='±2 std'
)
plt.legend()
plt.show()
```



mean values follow the data. the model performs well.

3.0.11 Regularization

The heteroskedastic model performs quite poor, explaining most of the variation in the data as noise. Since the noise term is too large, we can try to include regularization to shrink the weights and bias of the layer that models the standard deviation. In this case, we want to shrink the weights AND bias, as the bias controls the overall level of variance, which is estimated to be large in our previous model.

In this, case when defining our activation function `keras.layers.Dense`, we will use the options: - `bias_regularizer` - `kernel_regularizer`

Available regularizers are provided in: https://keras.io/api/layers/regularizers/

3.0.12 Exercise 8 (CORE)

Modify your previous model to include L_2 regularization for `output_sigma` layer. Use a penalty parameter of 1.0 for the bias and 0.01 for the weights.

Train the model for as many epochs as needed (when early stopping exits), and plot the mean function, also with lines for +/- 2*standard deviation, overlaid on top of a scatterplot of the data.

How does the model perform?

Note: if trained long enough, you should reach an R2 around 0.2 on the training data. If it stops before then, it has been initialized poorly and you should rerun the initial model definition to reinitialize with the weights and biases.

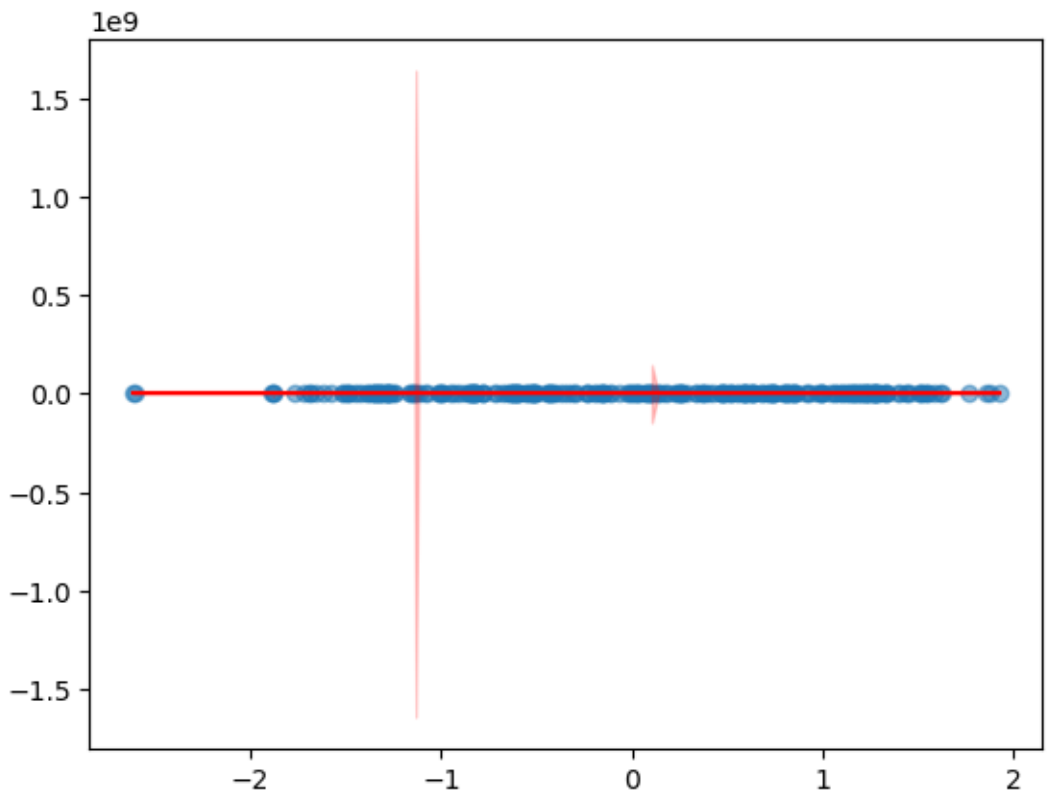
[65]:
```
# Defining the model with regularization
from tensorflow import keras
from tensorflow.keras import layers, regularizers

inputs = keras.Input(shape=(X.shape[1],))
x = layers.Dense(20, activation='relu')(inputs)
x = layers.Dense(10, activation='relu')(x)
output_mu = layers.Dense(1, name='mu')(x)
output_sigma = layers.Dense(
    1,
    name='sigma',
    kernel_regularizer=regularizers.l2(0.01),   # weights
    bias_regularizer=regularizers.l2(1.0)       # bias
)(x)
def nll_loss(y_true, y_pred):
    mu = y_pred[:, 0]
    log_sigma = y_pred[:, 1]
    sigma = tf.exp(log_sigma)
    return tf.reduce_mean(
        0.5 * tf.math.log(2 * np.pi) +
        log_sigma +
        0.5 * tf.square((y_true - mu) / sigma)
    )
output = layers.Concatenate()([output_mu, output_sigma])
model = keras.Model(inputs=inputs, outputs=output)
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=nll_loss
)
```

[66]:
```
# Fit the model
callback = keras.callbacks.EarlyStopping(monitor='val_loss', min_delta = 0.01,␣
 ↪mode = 'min',
```

```
                                                patience=10, restore_best_weights=True)

history = model.fit(
    X, y,
    epochs=300,
    batch_size=20,
    validation_split=0.1,
    callbacks=[callback],
    verbose=0
)
pred = model.predict(X)

mu_pred = pred[:, 0]
sigma_pred = np.exp(pred[:, 1])
mu_pred = mu_pred.flatten()
```

```
9/9              0s 7ms/step
```

[67]:
```
# Compute predictions and plot
idx = np.argsort(X[:,0])

X_sorted = X[idx]
y_sorted = y[idx]
mu_sorted = mu_pred[idx]
sigma_sorted = sigma_pred[idx]
plt.figure()
plt.scatter(X[:,0], y, alpha=0.5)

plt.plot(X_sorted[:,0], mu_sorted, color='red')
plt.fill_between(
    X_sorted[:,0],
    mu_sorted - 2*sigma_sorted,
    mu_sorted + 2*sigma_sorted,
    color='red',
    alpha=0.2
)
plt.show()
```



this model perform not very well.

When can of course tune and tweak the model further to try to get better performance. This can take some time. Instead, we will look at another dataset to understand how to wrap our keras model into sklearn's pipeline.

4 Part 2: Hitters Data

In the second part of this workshop, we will work with the hitters data, which was used in the Introduction to Statistical Learning (2023). The data records a number of statistics and salaries for baseball players. We wish to predict a baseball player's salary on the basis of various statistics associated with performance in the previous year.

In this part of the workshop, we will understand how to wrap our keras model into sklearn's pipeline to include our feature engineering steps.

Let's start by loading the data and looking at some basic information to understand what feature engineering steps are needed.

[25]:
```
hitters = pd.read_csv('Hitters.csv')
```

[26]:
```
hitters.head()
```

[26]:

```
   AtBat  Hits  HmRun  Runs  RBI  Walks  Years  CAtBat  CHits  CHmRun  CRuns  \
0    293    66      1    30   29     14      1     293     66       1     30
1    315    81      7    24   38     39     14    3449    835      69    321
2    479   130     18    66   72     76      3    1624    457      63    224
3    496   141     20    65   78     37     11    5628   1575     225    828
4    321    87     10    39   42     30      2     396    101      12     48

   CRBI  CWalks League Division  PutOuts  Assists  Errors  Salary NewLeague
0    29      14      A        E      446       33      20     NaN         A
1   414     375      N        W      632       43      10   475.0         N
2   266     263      A        W      880       82      14   480.0         A
3   838     354      N        E      200       11       3   500.0         N
4    46      33      N        E      805       40       4    91.5         N
```

[27]:

```
hitters.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 322 entries, 0 to 321
Data columns (total 20 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   AtBat      322 non-null    int64
 1   Hits       322 non-null    int64
 2   HmRun      322 non-null    int64
 3   Runs       322 non-null    int64
 4   RBI        322 non-null    int64
 5   Walks      322 non-null    int64
 6   Years      322 non-null    int64
 7   CAtBat     322 non-null    int64
 8   CHits      322 non-null    int64
 9   CHmRun     322 non-null    int64
 10  CRuns      322 non-null    int64
 11  CRBI       322 non-null    int64
 12  CWalks     322 non-null    int64
 13  League     322 non-null    object
 14  Division   322 non-null    object
 15  PutOuts    322 non-null    int64
 16  Assists    322 non-null    int64
 17  Errors     322 non-null    int64
 18  Salary     263 non-null    float64
 19  NewLeague  322 non-null    object
dtypes: float64(1), int64(16), object(3)
memory usage: 50.4+ KB
```

There are 20 variables in total, with 3 of them being of type object. We will need to do some encoding of these features.

We see that there are some missing values in salaries of players. Since this is our target variables, we will drop these missing values. Although it would be interesting to check if those who are missing

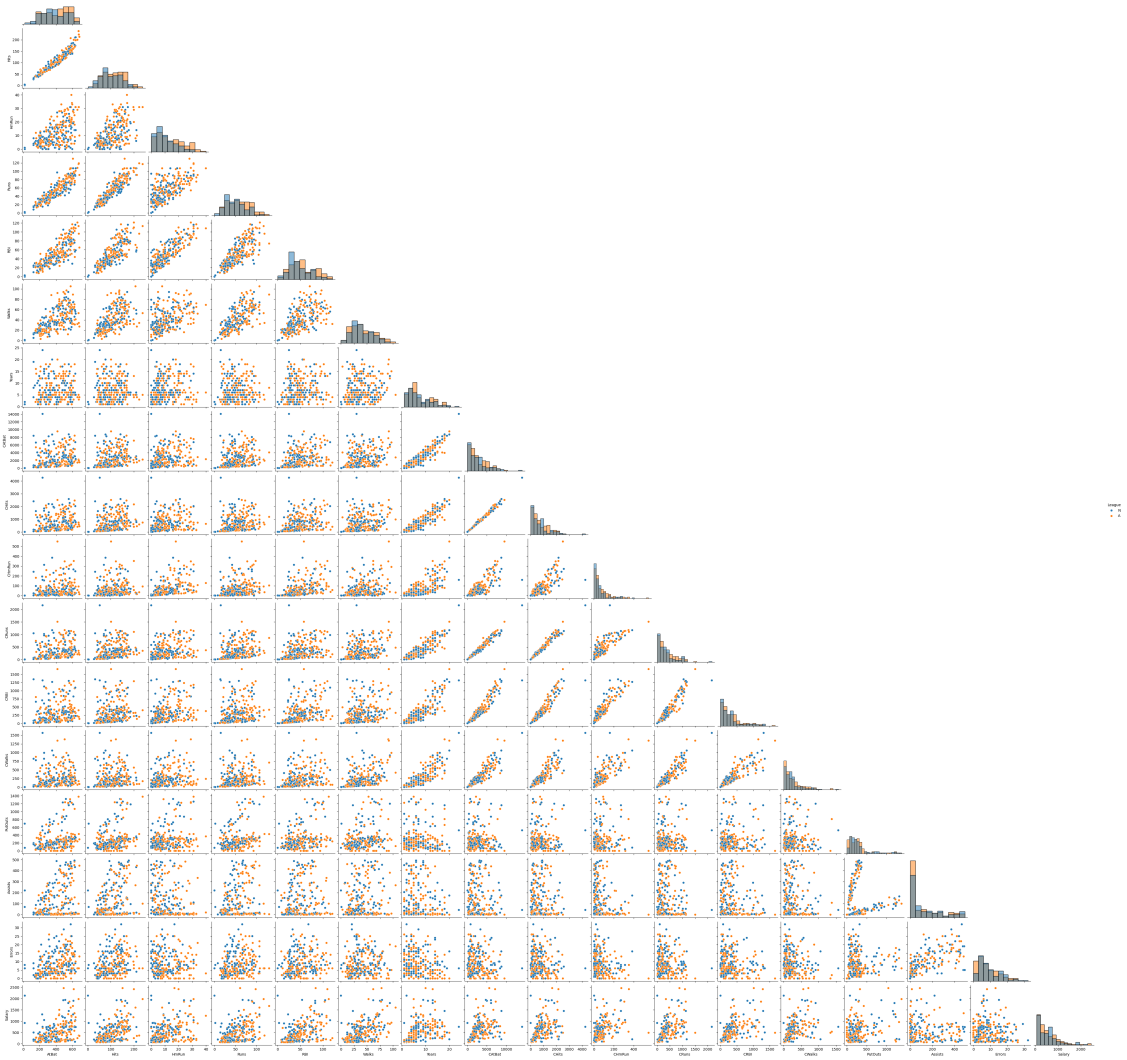
salaries have different characteristics in terms of their features (*not pursued here*); this might signal an informative missing mechanism and that our predictive model may only be applicable to a subpopulation of baseball players.

[28]:
```
# Remove observations with missing target variable
hitters = hitters.dropna()
N = hitters.shape[0]
D = hitters.shape[1]-1
print('Number of observations:', N)
print('Number of features:', D)
```

```
Number of observations: 263
Number of features: 19
```

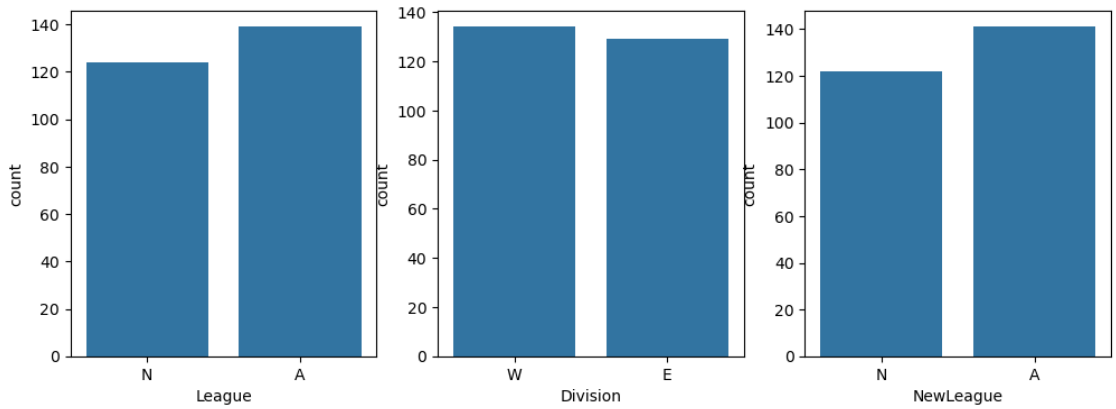
For the pairplot below, we observe high correlation among some of the features. Also since the numeric variables are on very different scales, we need to do some feature scaling.

[29]:
```
sns.pairplot(hitters, corner=True, diag_kind='hist', hue = 'League')
plt.show()
```



Checking the distribution of the categorical features, we see that they are all binary variables and there are no rare categories.

[30]:
```
fig, ax = plt.subplots(1,3, figsize = (12,4))
sns.countplot(hitters, x='League', ax =ax[0])
sns.countplot(hitters, x='Division', ax =ax[1])
sns.countplot(hitters, x='NewLeague', ax =ax[2])
plt.show()
```



Before we define our model, let's start by splitting into a training and test set.

[31]:
```
from sklearn.model_selection import train_test_split
y = hitters['Salary']
X = hitters.drop('Salary', axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,␣
↪random_state=1)
```

We are now ready to define our feature engineering steps to be used in our model pipeline. We will use standard scaling for the numerical variables and one hot encoding for the binary features.

[32]:
```
# Feature engineering steps
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder

# Names of numeric features
num_features = X_train.columns[X_train.dtypes=='int64']
# Names of categorical features
cat_features = X_train.columns[X_train.dtypes=='object']

preprocessor = make_column_transformer(
    (StandardScaler(), num_features),
    (OneHotEncoder(drop='first'),cat_features),
    verbose_feature_names_out=False
)
```

4.0.1 Baseline linear regression

We start by fitting a baseline linear regression model and investigating performance on the test data.

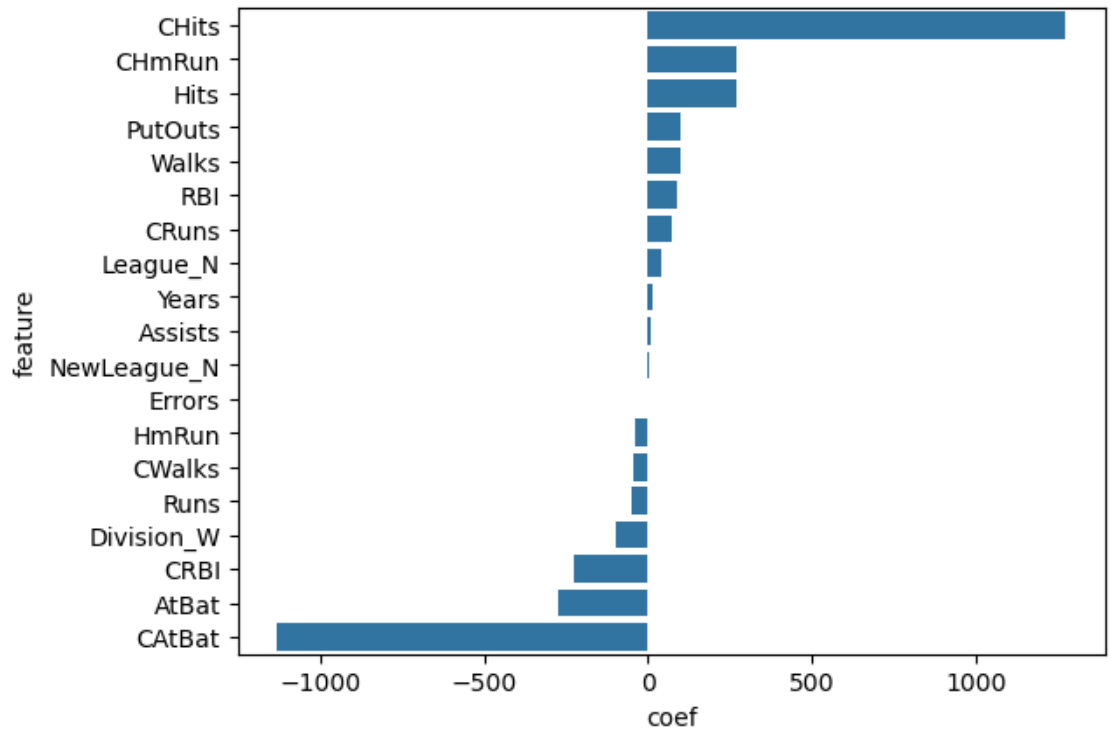
[33]:
```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression

lm = make_pipeline(
    preprocessor,
    LinearRegression()
).fit(X_train, y_train)
```

[34]:
```
from sklearn.metrics import root_mean_squared_error, mean_absolute_error,␣
↪r2_score

yhat_test = lm.predict(X_test)
print('R2:', r2_score(y_test, yhat_test))
print('RMSE:', root_mean_squared_error(y_test, yhat_test))
print('MAE:', mean_absolute_error(y_test, yhat_test))
```

```
R2: 0.32755419829816945
RMSE: 361.49290233703255
MAE: 262.16501067609096
```

The R2 is relatively low, but an advantage of the linear regression model is that we can examine the coefficients to understand the effect of the different features.

[35]:
```
coefs = pd.DataFrame({'feature': lm[:-1].get_feature_names_out(),
                      'coef': lm[-1].coef_}).sort_values ("coef",␣
↪ascending=False)
coefs
sns.barplot(coefs, y= 'feature', x = 'coef')
plt.show()
```



4.0.2 Wrapping keras model into an sklearn regression model

Our first step to turn our keras model into an sklearn regressor is to define the neural network model. We define a general MLP that can be constructed by passing in the desired number of neurons in each layer.

[ ]:
```
def mlp_model(X, y, lr = 0.001, layers=[64]):
    # Creates a basic MLP model
    D = X.shape[1]
    input_layer = keras.layers.Input(shape=(D,))

    hidden_layer = inputxr
    for layer_width in layers:
        hidden_layer = keras.layers.Dense(layer_width,␣
 ↪activation="relu")(hidden_layer)

    output_layer = keras.layers.Dense(1, activation='linear') (hidden_layer)

    model = keras.Model(input_layer, output_layer)
    model.compile(loss='mse', optimizer=keras.optimizers.Adam(learning_rate=lr),
                  metrics=[keras.metrics.R2Score(name='R2')])

    return model
```

Next, we simply use keras's sklearn wrappers: https://keras.io/api/utils/sklearn_wrappers/

This allows us to turn our keras model into an sklearn model. Through `model_kwargs`, we can specify the arguments of our `mlp_model` and through `fit_kwargs`, we can specify the arguments that we usually pass into keras's `.fit()`.

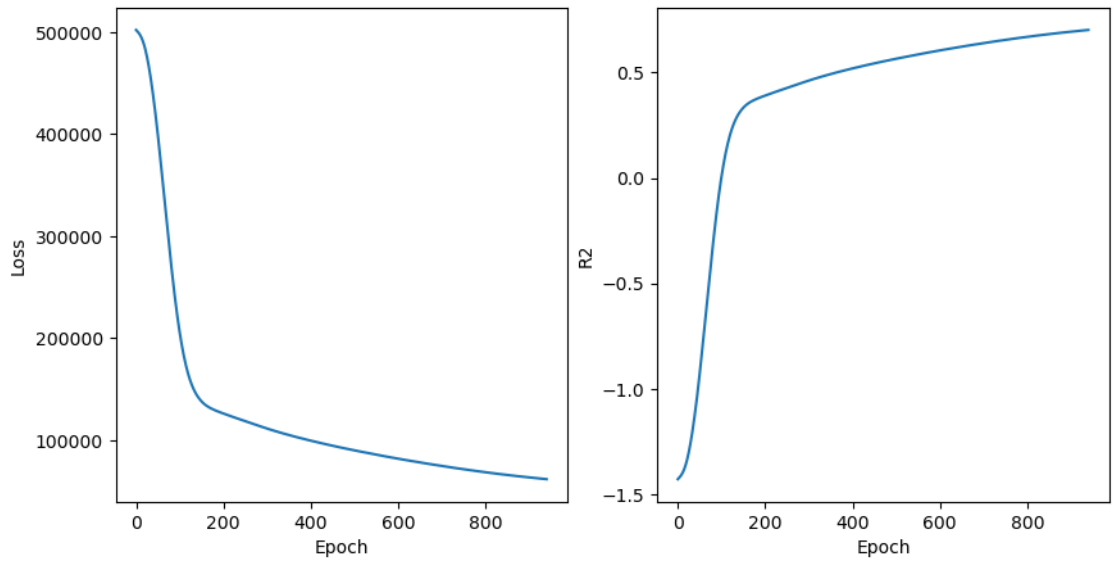
[37]:
```
from keras.wrappers import SKLearnRegressor

nn = SKLearnRegressor(
    model=mlp_model,
    model_kwargs={
        "layers": [64],
    },
    fit_kwargs={
        "epochs": 1000,
        "shuffle": True,
        "verbose": 0,
        "batch_size": 20,
        "callbacks": [keras.callbacks.EarlyStopping(monitor='R2', min_delta = 0.
 ↪001, mode = 'max',
                                                        patience=5,␣
 ↪restore_best_weights=True)]
    }
)
```

Now, that is a sklearn model, we can simply use pipelines to combine the preprocessing steps with our estimator!

[38]:
```
nn_pip = make_pipeline(
    preprocessor,
    nn
).fit(X_train, y_train)
```

Let's plot the history to check that we have trained the model for long enough.

[39]:
```
# Plot the training history
history =nn_pip[-1].model_.history.history
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].plot(history['loss'], label='Train Loss')
ax[0].set_xlabel('Epoch')
ax[0].set_ylabel('Loss')
ax[1].plot(history['R2'], label='Train R2')
ax[1].set_xlabel('Epoch')
ax[1].set_ylabel('R2')
plt.show()
```



Finally, let's check the performance of our model.

[40]:
```
yhat_test = nn_pip.predict(X_test)

print('R2:', r2_score(y_test, yhat_test))
print('RMSE:', root_mean_squared_error(y_test, yhat_test))
print('MAE:', mean_absolute_error(y_test, yhat_test))
```

```
WARNING:tensorflow:5 out of the last 11 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x7f4c6fa4ed40> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.
1/3              0s
35ms/stepWARNING:tensorflow:6 out of the last 13 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x7f4c6fa4ed40> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.
```

```
3/3              0s 25ms/step
R2: 0.5867707149736978
RMSE: 283.3783565705645
MAE: 189.40261504039543
```

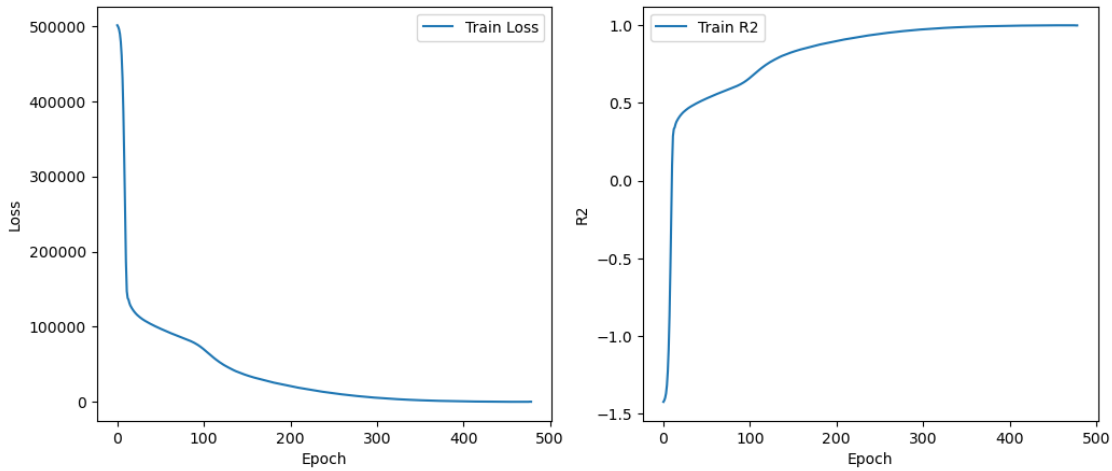
The performance is better than the linear model!

4.0.3 Exercise 9 (CORE)

Try training a deeper model, with your choice of depth and width.

How does the model perform?

[68]:
```
# Code for your answer here!
nn_deep = SKLearnRegressor(
    model=mlp_model,
    model_kwargs={
        "layers": [128, 64, 32],
    },
    fit_kwargs={
        "epochs": 1000,
        "shuffle": True,
        "verbose": 0,
        "batch_size": 20,
        "callbacks": [
            keras.callbacks.EarlyStopping(
                monitor='loss',
                patience=10,
                restore_best_weights=True
            )
        ]
    }
)
nn_deep_pip = make_pipeline(
    preprocessor,
    nn_deep
).fit(X_train, y_train)
history = nn_deep_pip[-1].model_.history.history
fig, ax = plt.subplots(1, 2, figsize=(12,5))
ax[0].plot(history['loss'], label='Train Loss')
ax[0].set_xlabel('Epoch')
ax[0].set_ylabel('Loss')
ax[0].legend()
ax[1].plot(history['R2'], label='Train R2')
ax[1].set_xlabel('Epoch')
ax[1].set_ylabel('R2')
ax[1].legend()

plt.show()
```



the model perform significantly better when use more epochs.

4.0.4 Exercise 10 (CORE)

Define a new MLP model, by altering the previous model to include L_1 regularization. You should define your model so that you can pass in different penalty parameters for each layer.

Now train you deeper model from Exercise 9, but with L_1 regularization for your choice of penalty parameters.

How does the model perform?

[ ]:
```
# Code for your answer here
from tensorflow.keras import regularizers

def mlp_model_L1(X, y, lr=0.001, layers=[128, 64, 32], l1_penalties=None):
    """
    X, y:
    lr:
    layers:
    l1_penalties:     L1
    """
    D = X.shape[1]
    input_layer = keras.layers.Input(shape=(D,))

    hidden_layer = input_layer

    if l1_penalties is None:
        l1_penalties = [0.0] * len(layers)

    for width, l1 in zip(layers, l1_penalties):
        hidden_layer = keras.layers.Dense(
```

```
            width,
            activation='relu',
            kernel_regularizer=regularizers.l1(l1)
        )(hidden_layer)

    output_layer = keras.layers.Dense(1, activation='linear')(hidden_layer)

    model = keras.Model(input_layer, output_layer)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss='mse',
        metrics=[keras.metrics.R2Score(name='R2')]
    )

    return model
    nn_L1 = SKLearnRegressor(
    model=mlp_model_L1,
    model_kwargs={
        "layers": [128, 64, 32],
        "l1_penalties": [1e-3, 1e-4, 1e-5]   #
    },
    fit_kwargs={
        "epochs": 1000,
        "shuffle": True,
        "verbose": 0,
        "batch_size": 20,
        "callbacks": [
            keras.callbacks.EarlyStopping(
                monitor='loss',
                patience=10,
                restore_best_weights=True
            )
        ]
    }
)
```

Type your answer here

4.0.5 Exercise 11 (EXTRA)

Define a new MLP model, but instead of using L_1 regularization, use dropout to help avoid overfitting. After defining each hidden layer, you need to add another line in your model, including a `keras.layers.Dropout` layer, see https://keras.io/api/layers/regularization_layers/dropout/

You should define your model so that you can pass in different dropout rates for each layer.

Now train you deeper model from Exercise 9, but with dropout for your choice of rates.

How does the model perform?

[43]:
```
# Code for your answer here
```

Type your answer here

5 Completing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

[44]:
```
!jupyter nbconvert --to pdf mlp_week10.ipynb
```

```
[NbConvertApp] Converting notebook mlp_week10.ipynb to pdf
[NbConvertApp] Support files will be in mlp_week10_files/
[NbConvertApp] Making directory ./mlp_week10_files
[NbConvertApp] Writing 118626 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 3526949 bytes to mlp_week10.pdf
```